

MES + SCADA + Gazebo Virtual Factory

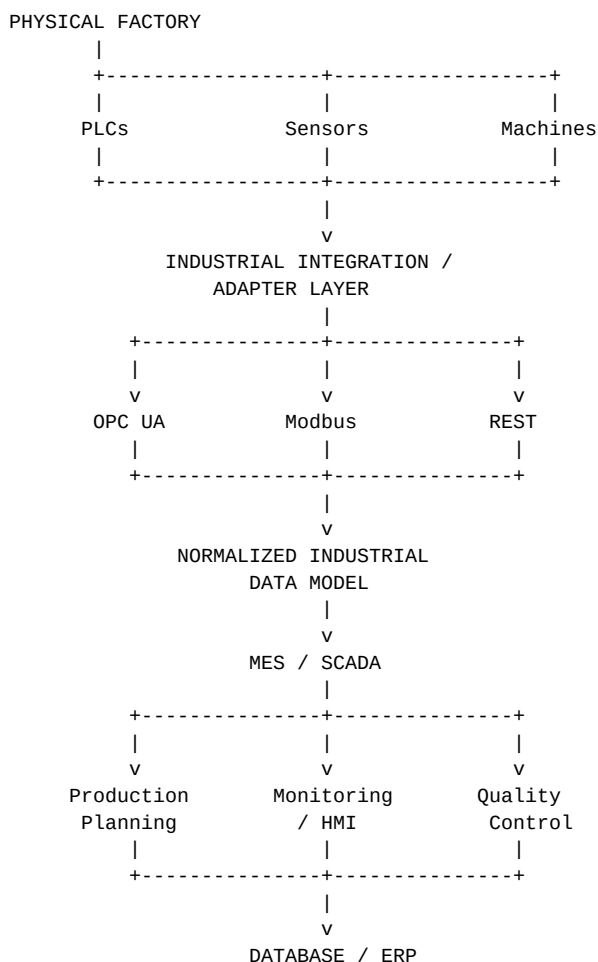
Source of Truth — Architecture, Phases, Industrial Integration & Development Guidance

Purpose: Preserve the project's architectural decisions, terminology, current implementation, and future direction so future development can resume without losing context.

Status: Living architecture document. Update this document whenever a major architectural decision is finalized.

1. Executive Architecture

The project is a virtual-factory-first development of an industrial MES + SCADA platform. Gazebo provides the simulated factory environment; C++ equipment plugins model/control simulated equipment; an Industrial Integration / Adapter Layer is intended to isolate the MES/SCADA software from vendor-specific industrial hardware and protocols; the application layer provides production, quality, maintenance, monitoring, authorization and other MES functions.



Key architectural principle: the Industrial Adapter is the boundary between heterogeneous factory equipment and our application. The MES should not have to understand every PLC vendor, sensor protocol, machine API or fieldbus implementation.

2. What We Are Building

Layer	Role in project
Virtual Factory	A Gazebo-based representation of a factory containing equipment such as conveyors and products.
Equipment Control	C++ Gazebo System plugins representing equipment behavior and control logic.
Industrial Adapter Layer	A future integration boundary that connects real PLCs, sensors and machines through multiple protocols.
MES	Production execution and management: orders, materials, operations, samples, quality feedback, production state, traceability, maintenance and related workflows.
SCADA/HMI	Operational monitoring and control views for plant personnel.
GUI	A web/.NET-oriented user interface is preferred over building a large desktop GUI directly in C++.
Security	Role-based authentication and authorization for management, supervisors, technicians and other plant users.

3. Distinct Development Phases

Phase 1 — Factory Foundation

Create the Gazebo world, floor, conveyor CV-001, product PRODUCT-001, correct physical dimensions/positions, model includes and basic visual materials.

Phase 2 — Equipment Plugin Foundation

Load libConveyorSystem.so, implement ConveyorSystem, identify the conveyor model and belt link, establish Configure and PreUpdate lifecycle.

Phase 3 — Conveyor Control

Implement Start(), Stop(), SetSpeed(), running state, speed state, development timing tests and diagnostic heartbeat.

Phase 4 — Product Motion

Detect PRODUCT-001 through the EntityComponentManager and update its Pose during conveyor operation so the simulated product moves with the conveyor.

Phase 5 — Industrial Equipment Abstraction

Move from a Gazebo-only mental model toward an equipment abstraction that can eventually be backed by real industrial adapters.

Phase 6 — Industrial Adapter Layer

Support protocol-specific connectors such as OPC UA, Modbus, EtherNet/IP, MQTT or vendor/API integrations while exposing normalized equipment data/commands to the application.

Phase 7 — MES Core

Production orders, work centers/equipment, material state, production execution, operator actions, samples, quality results, traceability and production feedback.

Phase 8 — SCADA / Operational HMI

Live equipment state, alarms, trends, commands, dashboards and role-specific operational screens.

Phase 9 — Security & Authorization

Authentication, role-based authorization, audit logging and least-privilege access.

Phase 10 — Real Factory Integration

Replace or augment simulation with real PLCs, sensors and machines through Industrial Adapters without rewriting the MES domain layer.

Phase 11 — Commercial Hardening

Configuration management, deployment, observability, testing, documentation, security hardening, backups and production support.

4. Current Gazebo Model

The current conveyor model is CV-001. Its frame is 4.0 m long, 1.2 m wide and 1.0 m high. The frame is centered at Z=0.5 m, giving a bottom of Z=0 and top of Z=1.0. The belt is 4.0 m × 1.0 m × 0.1 m and its

center is at $Z=1.05$ m, so its top is $Z=1.10$ m.

The product is $0.4\text{ m} \times 0.4\text{ m} \times 0.3\text{ m}$. Its half-height is 0.15 m . Therefore, when the belt top is $Z=1.10\text{ m}$, the product center is $Z=1.25\text{ m}$. The world places PRODUCT-001 at $X=-1.5$, $Y=0$, $Z=1.25$ so it begins near the conveyor start, centered across the belt and sitting on top.

```
factory.sdf
|
|-- floor
|
|-- CV-001
|
|-- PRODUCT-001

PRODUCT-001 initial pose:
<pose>-1.5 0 1.25 0 0 0</pose>

Pose fields:
X Y Z Roll Pitch Yaw
```

5. Static Conveyor Decision

The conveyor model was made static with `<static>true</static>`. This prevents the entire conveyor frame from physically falling or reacting as a dynamic rigid body. The product remains dynamic and can be moved by our control logic. This separated the machine structure from the simulated product motion.

6. Current C++ ConveyorSystem Architecture

```
ConveyorSystem
|
|-- entity_          -> CV-001 Gazebo model entity
|
|-- beltEntity_      -> "belt" link entity
|
|-- productEntity_   -> PRODUCT-001 entity
|
|-- name_            -> equipment identity
|
|-- running_         -> conveyor state
|
|-- speed_           -> conveyor speed (m/s)
|
|-- fault_           -> future fault state
|
|-- updateCount_     -> simulation update counter
|
|-- Configure()       -> discover model/link and initialize
|
|-- PreUpdate()       -> execute control every simulation update
|
|-- Start()           -> running=true; default speed=0.5 if needed
|
|-- Stop()            -> running=false; speed=0
|
|-- SetSpeed()        -> validate and store speed
```

7. Gazebo API / ECM Mental Model

Gazebo Sim exposes the simulation through entities and components. The EntityComponentManager (ECM) is the central mechanism used by the plugin to inspect and modify simulation state.

```
World
|
|-- Model entity: CV-001
|   |
|   |-- Link entity: frame
|   |
|   |-- Link entity: belt
```

```

|
+-- Model entity: PRODUCT-001
    |
    +-- Link: body

C++ plugin
|
+-- Model(entity)
|   |
|   +-- Name(ecm)
|   +-- LinkByName(ecm, "belt")
|
+-- ECM
    |
    +-- Name component -> discover PRODUCT-001
    +-- Pose component -> read/update product position
    +-- command/state components -> equipment control

```

A key lesson from the implementation: Gazebo Sim 8 exposes Link and its SetLinearVelocity API, but the required headers/API must match the installed version. When direct Link use caused a compilation problem, the installed headers were inspected to verify the available API. The build ultimately succeeded after correcting the ECM component update call.

8. Product Motion Logic

```
Each PreUpdate:
|
|-- paused? ---- yes --> return
|
|-- increment updateCount
|
|-- locate PRODUCT-001 if entity not known
|
|-- at update 1000 --> Start()
|
|-- at update 5000 --> Stop()
|
|-- if running:
|   |
|   |-- read PRODUCT-001 Pose
|   |-- dt = simulation time step
|   |-- X += speed * dt
|   |-- write Pose back through ECM
|
|-- set belt linear velocity
|
|-- every 100 updates --> diagnostic heartbeat
```

The 1000/5000 update Start/Stop logic is explicitly a development test, not the final production control architecture. Later, commands will originate from a control service, SCADA/HMI, MES workflow or industrial interface rather than hard-coded simulation counters.

9. Why the C++ Plugin Still Matters for a Real Factory

The C++ plugin is not intended to be the only software that talks to every real machine. Its primary current role is to model equipment behavior inside Gazebo and establish the equipment-control architecture. In a real deployment, the same higher-level equipment concepts—state, speed, start, stop, faults, telemetry, commands—can be implemented by an Industrial Adapter backed by real protocols.

VIRTUAL DEPLOYMENT

MES/SCADA

```
|
+--> Gazebo equipment interface
|
+--> ConveyorSystem C++ plugin
+--> simulated sensors
+--> simulated product
```

REAL DEPLOYMENT

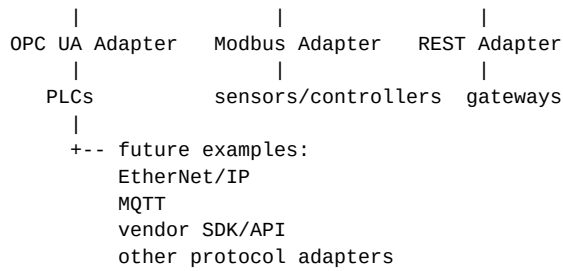
MES/SCADA

```
|
+--> Industrial Adapter
|
+--> OPC UA --> PLC
+--> Modbus --> sensor/controller
+--> EtherNet/IP --> PLC
+--> MQTT --> gateway/device
+--> REST --> vendor/custom system
```

10. Industrial Adapter — Why It Is Necessary

Factories are heterogeneous. Different equipment can use different PLC vendors, field protocols, gateways, APIs and data models. Therefore, the project should not make the MES directly dependent on a particular hardware vendor.

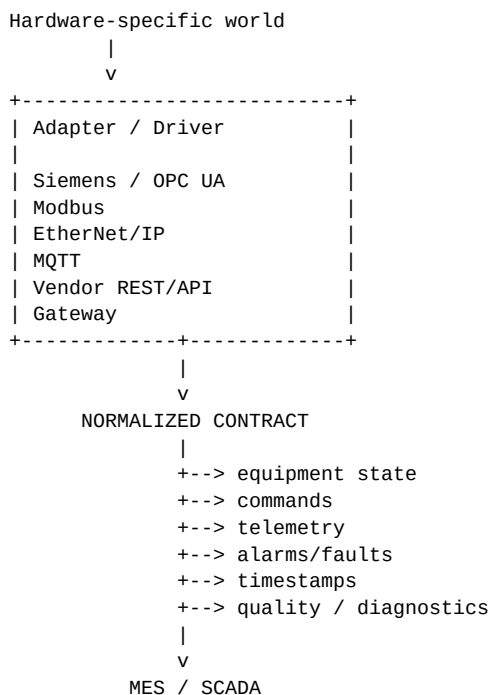
```
INDUSTRIAL ADAPTER
|
+-----+-----+
```



The adapter layer should translate protocol-specific data into a normalized internal equipment model. An unsupported or unusual device can be integrated through a gateway or its vendor API and exposed through REST rather than forcing the MES to understand the device's proprietary protocol.

11. Hardware Scalability Principle

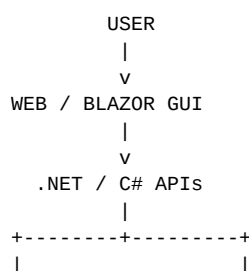
We should not attempt to write one universal driver for every piece of hardware. Instead, use a plug-in/adaptor architecture: each protocol or vendor family gets a connector, while the rest of the system consumes a common contract.

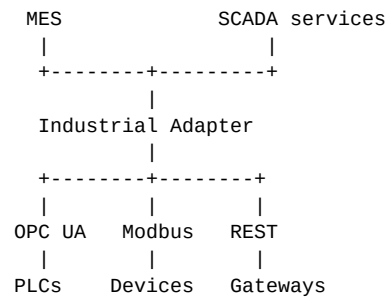


12. GUI Strategy — Why Not Build the Main GUI in C++

The project is intentionally separating equipment/control logic from the user interface. C++ remains appropriate for simulation plugins, high-performance industrial services and hardware-facing components. A web-oriented GUI is more productive for a large MES/SCADA application.

Preferred direction: .NET/C# backend services and a web UI, with Blazor as a strong candidate for the application interface. This keeps C++ where it provides the most value and lets the UI evolve independently.





13. MES Scope — Intended Functional Areas

MES area	Scope
Production Orders	Create/import orders, quantities, due dates, priorities, recipes/routings and execution status.
Production Execution	Dispatch work, start/stop operations, record quantities, scrap, downtime and operator actions.
Material Management	Raw-material availability, lot/batch tracking, consumption, reservations and sufficiency against production orders.
Quality Management	Sampling, laboratory/inspection results, pass/fail decisions, deviations, feedback and release/hold status.
Traceability	Link raw-material lots to work orders, machines, operators, samples and finished products.
Equipment Management	Equipment identity, state, operating parameters, availability and faults.
Maintenance	Preventive maintenance, corrective work, work requests, downtime and maintenance history.
Alarms & Events	Equipment alarms, acknowledgements, event history and escalation.
Performance	OEE-related metrics, cycle time, downtime, throughput, utilization and production KPIs.
Inventory / WIP	Work-in-progress location/state, material movement and finished goods state.
Operator Actions	Operator login, task assignment, confirmations and electronic records.
Reporting	Production, quality, material, downtime, maintenance and management reports.

14. Security / Users / Roles

The system will be used by different plant roles and therefore requires authentication plus role-based authorization. A useful terminology decision is to use Supervisor for the floor-management role rather than 'Floor Manager'.

```
Authentication
  |
  v
Identity / Login
  |
  v
Role + Permission evaluation
  |
  +--> Plant Manager
  +--> Production Manager
  +--> Supervisor
  +--> Technician
  +--> Quality Officer
  +--> Operator
  +--> Maintenance
  +--> System Administrator
  |
  v
Audit Log
```

Principle:

Authentication = "Who are you?"

Authorization = "What are you allowed to do?"

Audit = "What did you do, when, and where?"

15. Repository / Development Discipline

The project must preserve architectural context through a Source-of-Truth document and disciplined Git history. Before significant changes: inspect git status, understand the current commit, make the smallest logical change, build/test, then commit with a meaningful message.

```
Project root:
~/Documents/virtual-factory

Important current structure:
gazebo/
  models/
    conveyor/
      model.sdf
    product/
      model.sdf
  worlds/
    phase1/
      factory.sdf
  plugins/
    conveyor/
      ConveyorSystem.hh
      ConveyorSystem.cc
    build/

Git principle:
inspect -> change -> build -> test -> commit

Never assume a build directory exists.
For the conveyor plugin:
cd ~/Documents/virtual-factory/gazebo/plugins/conveyor/build
cmake --build .

Do not use:
cmake --build build
when already inside the build directory.
```

16. Current Known State / Milestone

At the point captured by this document, the conveyor plugin builds successfully. The conveyor can be started/stopped by the current development test logic, the belt is controlled through the Gazebo simulation API, PRODUCT-001 is discovered through the ECM and its pose is advanced in +X while the conveyor is running.

The project is now ready to move from the basic simulation mechanics toward cleaner equipment abstractions, industrial adapter design, MES domain modeling, SCADA/HMI architecture, and eventually real hardware integration.

17. Critical Decisions to Preserve

1. Gazebo is the virtual factory/simulation environment.
2. C++ is used for Gazebo equipment plugins and suitable industrial/backend components—not as the primary large MES GUI technology.
3. The Industrial Adapter is the boundary between real factory hardware and MES/SCADA.
4. The adapter layer must support multiple protocols rather than assuming OPC UA is universal.
5. REST is an important escape/integration path for unsupported or custom equipment through gateways/vendor systems.
6. MES should consume normalized equipment data and commands instead of vendor-specific hardware APIs.

7. The main GUI should be web-oriented; .NET/C# and Blazor are preferred candidates.
8. The product and conveyor geometry/height calculations are deliberate and should be preserved unless the physical design changes.
9. Development test timers such as update 1000/5000 are temporary and must not become the final industrial control mechanism.
10. Security must include authentication, authorization, role-based permissions and auditing.
11. Supervisor is the preferred term for the floor-management role.
12. The architecture should eventually support production orders, raw-material sufficiency, sampling, quality feedback, traceability, maintenance, alarms, KPIs and reporting.

18. Next Architectural Direction

```

CURRENT
Gazebo
|
ConveyorSystem C++
|
ECM
|
simulated product movement
    |
    v

NEXT
Equipment abstraction
|
Industrial Adapter interfaces
|
+-- OPC UA
+-- Modbus
+-- EtherNet/IP
+-- MQTT
+-- REST/vendor API
|
Normalized equipment contract
|
.NET/C# application services
|
+-- MES
+-- SCADA
+-- Authentication / Authorization
+-- Audit
|
Blazor/Web GUI
|
Users: Managers, Supervisors, Technicians, Operators,
      Quality, Maintenance, Administrators

```

Document intent: This is the project's architectural memory. When future implementation choices conflict with this document, first identify whether the Source of Truth should be deliberately revised; do not silently drift the architecture.